

Capitolo 12 - Sistemi di I/O

Status Done

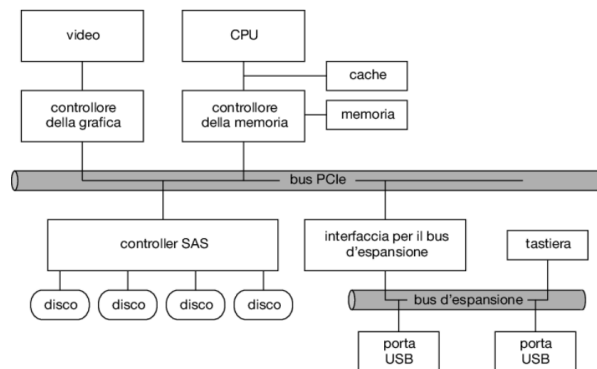
Il compito del SO è gestire e controllare i dispositivi di I/O, che variano enormemente per funzione e velocità (dal mouse al nastro magnetico).

- **Sottosistema di I/O del kernel:** è la parte del SO che isola il kernel dalla complessità dei dispositivi.
- **Uniformità vs. Varietà:** esiste una tendenza verso l'uso di interfacce standard, ma la continua invenzione di nuovi tipi di HW rende difficile l'integrazione.
- **Driver dei dispositivi:** per risolvere la diversità, il kernel utilizza moduli software chiamati *driver*. Questi nascondono le particolarità dell'HW e offrono al SO un'interfaccia uniforme.

Hardware di I/O

I dispositivi comunicano con il computer tramite segnali. I concetti base della connessione sono:

- **Porta:** il punto di connessione fisica (es. porta seriale).
- **Bus:** un insieme di fili e un protocollo di messaggi condiviso tra più dispositivi.
 - **PCIe:** il bus di sistema moderno, veloce e flessibile, basato su "canali" (es. x8, x16) full-duplex.
- **Daisy chain:** una catena di dispositivi collegati uno dopo l'altro che termina in una porta del computer, funzionando logicamente come un bus.
- **Controllore:** l'elettronica che gestisce porta, bus o dispositivo. Può essere una scheda separata (**HBA - Host Bus Adapter**) o integrata nel dispositivo stesso (come nei dischi SATA).



I/O Memory Mapped

Esistono due modi principali con cui la CPU comunica con i registri del controllore:

1. **Istruzioni speciali di I/O:** la CPU usa comandi specifici per inviare dati a determinati indirizzi di porta.
2. **I/O Memory Mapped:** tecnica oggi prevalente. I registri del dispositivo sono mappati direttamente nello **spazio di indirizzamento della memoria fisica**. La CPU scrive sul dispositivo usando le normali istruzioni di memoria, il che è molto più veloce per grandi volumi di dati (es. schede grafiche).

Un controllore standard ha solitamente quattro registri:

- **Status** indica se il dispositivo è occupato, se ci sono dati pronti o se si è verificato un errore.
- **Control:** scritto dalla CPU per impartire comandi o cambiare modalità.
- **Data-in / Data-out:** per il passaggio effettivo dei byte.

Polling (interrogazione ciclica)

È la forma più semplice di interazione (**Handshaking**):

- la CPU legge ripetutamente il registro *Status* finché il bit **busy** non diventa 0.

- **Inefficienza:** se il dispositivo è lento, la CPU spreca cicli in **attesa attiva (busy-waiting)**. È utile solo per dispositivi estremamente veloci.

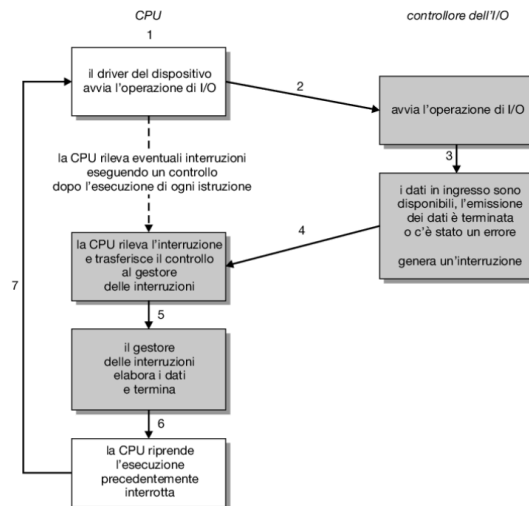
Interrupt

Il meccanismo delle interruzioni permette alla CPU di rispondere a eventi asincroni senza fare polling.

1. Il controllore invia un segnale sulla **linea di richiesta dell'interruzione**.
2. La CPU, dopo ogni istruzione, controlla questa linea. Se attiva, salva lo stato corrente e salta alla **routine di gestione dell'interruzione (Interrupt Handler)**.
3. Completato il compito, la CPU riprende il lavoro precedente.

Caratteristiche avanzate:

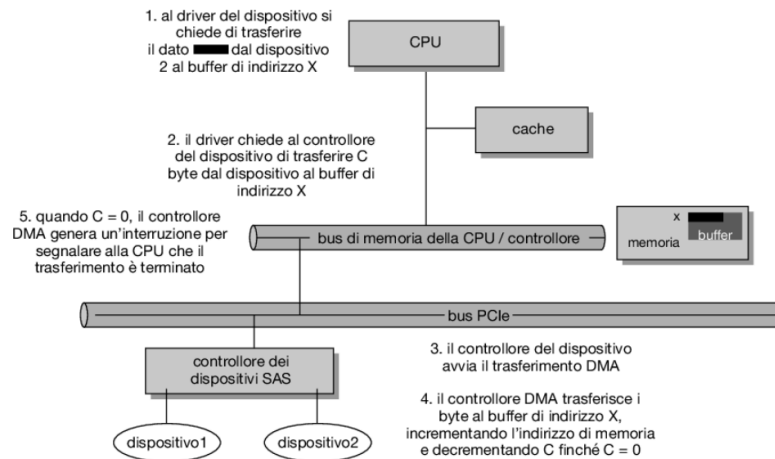
- **Interruzioni mascherabili:** possono essere disattivate durante fasi critiche.
- **Interruzioni non mascherabili:** per eventi gravissimi (es. errori di memoria).
- **Vettore delle interruzioni:** una tabella di indirizzi che punta ai diversi gestori per evitare di cercarli ogni volta.
- **Interrupt chaining:** quando più dispositivi condividono lo stesso numero di interruzione, il vettore punta a una lista di gestori.
- **Priorità:** permette di servire prima gli eventi più urgenti (es. errore HW prima di una chiamata di sistema).
- **Gestione in due fasi:** spesso divisa in **FLIH** (Primo livello, veloce, salva lo stato) e **SLIH** (Secondo livello, esegue il lavoro pesante).



Accesso diretto alla memoria (DMA)

Per evitare che la CPU si occupi di spostare ogni singolo byte (operazione chiamata **PIO - Programmed I/O**), si usa il **controllore DMA**.

- **Funzionamento:** la CPU fornisce al DMA l'indirizzo sorgente, quello di destinazione e la quantità di dati, poi si dedica ad altro. Il DMA gestisce il trasferimento direttamente tra dispositivo e memoria.
- **Sottrazione di cicli (Cycle Stealing):** quando il DMA usa il bus, la CPU non può accedere alla memoria centrale (ma può continuare a usare la sua cache).
- **DVMA:** versione virtuale del DMA che permette trasferimenti diretti tra due dispositivi memory-mapped senza passare dalla memoria centrale.



Concetti principali dell'hardware di I/O

Per comprendere l'I/O dal punto di vista del SO, i pilastri sono:

- **bus e controllori;**
- **registri di interfaccia** (status, control, data);
- **handshaking** tramite **polling** o **interruzioni;**
- **delega al DMA** per i trasferimenti massivi.

Interfaccia di I/O delle applicazioni

Per gestire la complessità, i SO utilizzano astrazione, incapsulamento e stratificazione.

- **Astrazione:** le differenze specifiche tra i dispositivi sono raggruppate in classi generali.
- **Driver dei dispositivi:** sono moduli del kernel che nascondono i dettagli HW al sottosistema di I/O. Ogni SO ha i propri standard driver (un driver per Windows non funziona su Linux).
- **Major e Minor Number (Unix/Linux):**
 - **Major Number:** identifica il tipo di dispositivo (e quindi quale driver chiamare).
 - **Minor Number:** identifica la specifica istanza o partizione di quel dispositivo.

Caratteristiche dei dispositivi

I dispositivi vengono classificati in base a diverse dimensioni:

1. **Trasferimento:** a blocchi (dischi) o a flusso di caratteri (tastiere).
2. **Accesso:** sequenziale (nastri) o diretto (dischi).
3. **Sincronia:** sincroni (tempo di risposta prevedibile) o asincroni (irregolari).
4. **Condivisione:** un dispositivo condivisibile può essere usato in modo concorrente da diversi processi o thread, impossibile se il dispositivo è dedicato.
5. **Direzione:** lettura/scrittura, sola lettura o sola scrittura.

Back door (`ioctl`): poiché le chiamate standard non coprono ogni funzione possibile, Unix fornisce `ioctl` (I/O Control), che permette alle applicazioni di inviare comandi arbitrari direttamente ai driver.

Dispositivi a blocchi e a caratteri

- **Dispositivi a blocchi:** vi si accede tramite interfacce di file system con comandi `read()`, `write()` e `seek()`.
 - **Raw I/O (I/O a basso livello):** permette ad applicazione come i database di bypassare il file system e gestire direttamente i blocchi, evitando ridondanze nei buffer e nei lock.
 - **I/O Mappato in memoria:** permette di accedere a un file come se fosse un vettore di byte nella RAM. È molto efficiente perché sfrutta il meccanismo della memoria virtuale.
- **Dispositivi a caratteri:** (Es. tastiere, mouse). Utilizzano funzioni `get()` e `put()`. Spesso sono gestiti con librerie che offrono buffering ed editing riga per riga.

Dispositivi di rete

L'I/O di rete è troppo diverso dai dischi per usare `read/write` standard. L'interfaccia universale è la **socket**.

- **Concetto:** funziona come una "presa" a cui connettersi.
- **Funzioni chiave:** creazione, connessione a un indirizzo remoto e ascolto di connessioni locali.
- `select()`: una funzione che permette a un server di monitorare molte socket contemporaneamente, eliminando la necessità di polling o attesa attiva.

Orologi e timer

Forniscono tre funzioni: ora corrente, tempo trascorso e timer programmabili.

- **Timer programmabile:** genera interruzioni periodiche. Usato dallo scheduler per i quanti di tempo o per il flushing della cache.
- **HPET (High Precision Event Timer):** presente nei PC moderni, offre alta risoluzione ma può soffrire di **deriva** (allontanamento dall'ora esatta), corretta via rete tramite protocolli come NTP.

I/O non bloccante e asincrono

- **I/O bloccante:** l'applicazione si sospende finché l'operazione non termina. È il modello più semplice per il programmatore.
- **I/O non bloccante:** la chiamata restituisce immediatamente quanto è stato possibile trasferire (anche nulla), senza sospendere il thread.
- **I/O asincrono:** l'applicazione avvia la richiesta e continua subito il lavoro. Il sistema avviserà l'app a operazione conclusa tramite una **callback**, un segnale o una variabile.

I/O vettorizzato (Scatter-Gather)

Questa tecnica permette di eseguire più operazioni di I/O con una sola chiamata di sistema.

- **Scatter (semina):** legge dati da un dispositivo e li distribuisce in più buffer separati.
- **Gather (raccolta):** prende dati da più buffer e li scrive in un'unica operazione sul dispositivo.
- **Vantaggi:** riduce i cambi di contesto (overhead) ed evita la copia manuale dei dati in un unico buffer contiguo. Spesso garantisce l'atomicità dell'operazione.

Sottosistema di I/O del kernel

Il sottosistema di I/O del kernel funge da mediatore tra le applicazioni e l'HW, offrendo una serie di servizi fondamentali per l'efficienza, la correttezza e la protezione del sistema.

Scheduling dell'I/O

Lo scheduling consiste nel determinare l'ordine ottimale di esecuzione delle richieste di I/O, poiché l'ordine di arrivo raramente è il più efficiente.

- **Obiettivi:** migliorare il throughput, ridurre i tempi di attesa e garantire equità tra i processi.
- **Esempio:** su un disco rigido, lo scheduler riordina le richieste per minimizzare il movimento della testina (es. servendo in ordine numerico crescente invece che casuale).
- **Meccanismo:** il kernel mantiene una coda di richieste per ogni dispositivo. Per l'I/O asincrono, si usa una tabella dello stato dei dispositivi per monitorare quali unità sono occupate, inattive o malfunzionanti.

Gestione dei buffer

Un buffer è un'area di memoria usata per memorizzare dati durante il trasferimento.

- **Motivi d'uso:**
 1. **Disaccoppiamento di velocità:** gestisce la differenza tra produttore e consumatore (es. modem lento e disco veloce). Si usa spesso la **doppia bufferizzazione** per permettere al modem di scrivere nel secondo buffer mentre il primo viene svuotato su disco.
 2. **Adattamento dei dati:** gestisce dispositivi che usano blocchi di dimensioni diverse (comune nella frammentazione/riassemblaggio dei pacchetti di rete).
 3. **Semantica della copia:** garantisce che la `write()` scriva i dati così come erano al momento della chiamata, copiandoli nel kernel prima di restituire il controllo dell'app, proteggendoli da modifiche successive.

Cache

A differenza del buffer (che può contenere l'unica copia dei dati), la cache mantiene copie di dati esistenti altrove per velocizzarne l'accesso.

- **Funzionamento:** il kernel controlla se i dati richiesti sono già nella cache; in caso positivo, evita l'accesso fisico al dispositivo lento.
- **Integrazione:** spesso una stessa regione di memoria funge sia da buffer che da cache (es. i buffer di pagina del file system).

Code di spooling e riservazione dei dispositivi

Lo **spooling** gestisce dispositivi che non possono alternare dati da diverse sorgenti (es. stampanti).

- **Funzionamento:** i dati di ogni processo vengono scritti in file temporanei su disco. Solo quando il file è completo, viene accodato per la stampa sequenziale.
- **Riservazione esclusiva:** alcuni sistemi permettono a un processo di riservare un dispositivo (es. unità a nastro) tramite un'apertura esclusiva, impedendo ad altri l'accesso fino al rilascio.

Gestione degli errori

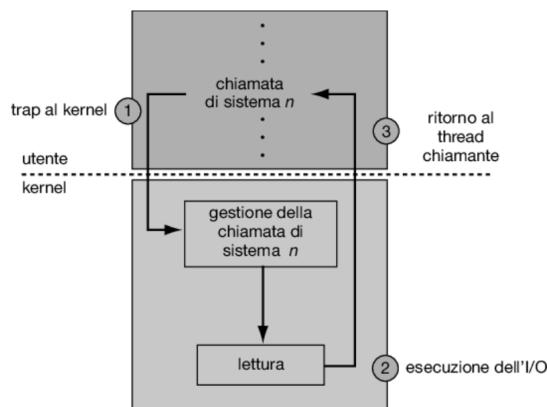
Il SO deve gestire malfunzionamenti HW o di rete senza bloccare l'intero sistema.

- **Errori temporanei:** gestiti tramite tentativi automatici (retry).
- **Segnalazione:** le chiamate di sistema restituiscono bit di stato; Unix usa la variabile `errno`. Protocolli complessi come SCSI forniscono codici dettagliati (*sense keys*).

Protezione dell'I/O

Per prevenire danni, tutte le istruzioni di I/O sono **privilegiate**.

- **Accesso mediato:** l'utente deve invocare una chiamata di sistema; il kernel verifica i permessi prima di agire.
- **Indirizzi protetti:** gli indirizzi mappati in memoria (memory-mapped) devono essere protetti dal sistema di gestione della memoria. In casi speciali (es. grafica ad alte prestazioni), il kernel può delegare l'accesso diretto a sezioni specifiche tramite lock.



Strutture dati del kernel

Il kernel mantiene strutture interne per tracciare lo stato dell'I/O (file aperti, socket, stati dei driver).

- **Approccio Unix:** usa una tecnica orientata agli oggetti dove un record di "file aperto" contiene puntatori a funzioni specifiche per il tipo di oggetto (file, disco raw, socket), permettendo chiamate `read()` uniformi.
- **Approccio Windows:** basato su uno scambio di messaggi che attraversano i vari strati del sottosistema di I/O.

Gestione energetica

L'efficienza energetica è critica sia nei data center (riduzione calore e costi) sia nei dispositivi mobili (durata batteria).

- **Android - Tre tecniche chiave:**
 1. **Power collapse (collasso di potenza):** spegne quasi tutto il dispositivo portando la CPU allo stato di sospensione minimo, pur restando in grado di "svegliarsi" per un'interruzione.

2. **Component-Level Management:** android usa un albero topologico dei dispositivi. Se un componente (es. audio) non è usato, viene spento. Se tutti i nodi di un ramo sono inattivi, si spegne l'intero ramo (es. bus di sistema).
 3. **Wavelocks:** meccanismo che permette alle app di impedire temporaneamente il Power Collapse (es. durante il download di un aggiornamento o la visione di un video).
- **Standard ACPI:** nei PC moderni, il kernel usa il firmware ACPI per rilevare e gestire gli stati energetici dei dispositivi in modo standardizzato.

Concetti principali del sottosistema di I/O del kernel

In sintesi, il sottosistema coordina:

- spazio dei nomi e controllo accessi;
- gestione di buffer, cache e spool;
- scheduling delle richieste;
- gestione degli errori, protezione e gestione energetica.

Trasformazione delle richieste di I/O in operazioni hardware

Il SO deve collegare i nomi dei file usati dalle applicazioni agli indirizzi fisici dei dispositivi.

- **Associazione nome-dispositivo:**
 - **MS-DOS:** usa un prefisso separato da due punti (es. `C:`). Una tabella interna associa il prefisso all'indirizzo hardware del controller.
 - **UNIX:** i nomi dei dispositivi sono integrati nel file system. Il sistema usa una tabella di montaggio (mount table) per associare un percorso a un dispositivo.
- **Major e Minor Number:** in UNIX, un dispositivo è identificato da una coppia di numeri. Il major number seleziona il driver corretto; il minor number viene passato al driver per identificare lo specifico HW (es. quale porta o indirizzo memory-mapped usare).
- **Flessibilità:** l'uso di tabelle lookup permette di aggiungere nuovi dispositivi o caricare driver su richiesta senza dover ricompilare il kernel.

Ciclo di una richiesta di lettura bloccante (Esempio)

1. **Chiamata di sistema:** l'app chiama `read()`.
2. **Kernel check:** il kernel controlla i permessi e verifica se i dati sono già nella **buffer cache**.
3. **Scheduling:** se i dati non ci sono, il processo viene sospeso e la richiesta inserita nella coda del dispositivo.
4. **Driver & Hardware:** il driver scrive nei registri del controller, il controller aziona l'hardware.
5. **DMA e interrupt:** i dati vengono trasferiti (spesso via DMA). Al termine, il controller genera un interrupt.
6. **Ripresa:** il gestore dell'interruzione avvisa il driver, il kernel copia i dati nello spazio dell'app e sposta il processo nella coda dei pronti.

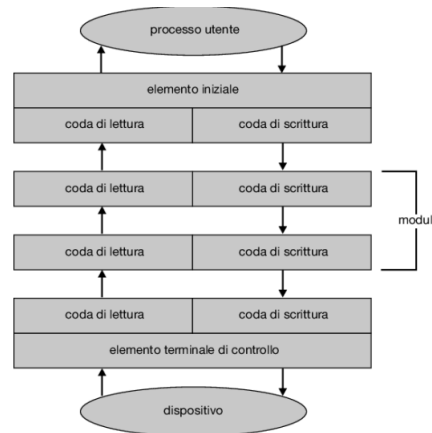
STREAMS (Unix System V)

STREAMS è un meccanismo che permette di comporre dinamicamente moduli software tra un processo utente e un driver.

- **Struttura:**
 - **Stream Head:** estremità vicina all'utente. Riceve le chiamate di sistema e le trasforma in messaggi.
 - **Driver End:** estremità vicina all'hardware. Comunica direttamente con il dispositivo fisico.
 - **Stream Modules:** questi sono i "pezzi di tubo" intermedi. Possono essere aggiunti o rimossi dinamicamente. Ogni modulo fa un lavoro specifico (es. uno cripta i dati, uno controlla gli errori, ecc...).
- Ogni pezzo della catena (Head, Moduli, Driver) ha due code:
 1. **Coda di scrittura (downstream):** i dati scendono dall'applicazione verso il dispositivo.
 2. **Coda di lettura (Upstream):** i dati salgono dal dispositivo verso l'applicazione.

Il trasferimento avviene tramite **scambio di messaggi**. Un modulo non chiama direttamente il successivo, ma deposita un messaggio nella sua coda.

- **Controllo di flusso:** per evitare che una coda veloce inondi una lenta, i messaggi vengono bufferizzati; se il buffer è pieno, il modulo smette di accettare dati, bloccando la catena fino alla sorgente.
- **Vantaggi:** modularità estrema. Un modulo per un protocollo di rete può essere usato sia con schede Ethernet che wireless.



Prestazioni

L'I/O è un collo di bottiglia critico. Gestire migliaia di interruzioni e cambi di contesto al secondo consuma molti cicli di CPU e invalida la cache.

- **Overhead della rete:** una semplice comunicazione (es. digitare un carattere in remoto) causa un numero enorme di interrupt, passaggi tra kernel/utente e pacchetti di rete su entrambe le macchine.
- **Soluzioni hardware:**
 - **Canali di I/O:** CPU specializzate (comuni nei mainframe) che gestiscono l'I/O lasciando libera la CPU principale.
 - **Concentratori di terminali:** gestiscono centinaia di connessioni lente su un'unica porta veloce.
- **Principi di ottimizzazione:**
 1. Ridurre i cambi di contesto e le copie di dati.
 2. Usare il polling invece degli interrupt se l'attesa è molto breve.
 3. Sfruttare il DMA e controllori intelligenti per scaricare la CPU.
 4. Bilanciare le prestazioni tra CPU, memoria e bus per evitare "colli di bottiglia".

Evoluzione dei servizi di I/O

Ci si può chiedere se i servizi di i/o si debbano implementare nei dispositivi hardware, nei loro driver, o nelle applicazioni.

1. **Applicazione:** la funzione nasce come un normale programma. È facile da modificare e sicura, ma è lenta perché deve continuamente chiedere permessi al sistema.
2. **Kernel:** quando la funzione diventa uno standard, viene inserita nel SO. Diventa molto più veloce perché accede direttamente alle risorse, ma se ha un errore può far crashare l'intero computer.
3. **Hardware:** infine, la funzione viene inserita nel chip. È la soluzione più potente in assoluto e non pesa sulla CPU, ma non è più modificabile: se vuoi una versione nuova, devi cambiare il pezzo fisico.